EWB COURSES

CLOUD LABS / STUDENT PROJECT REPORT

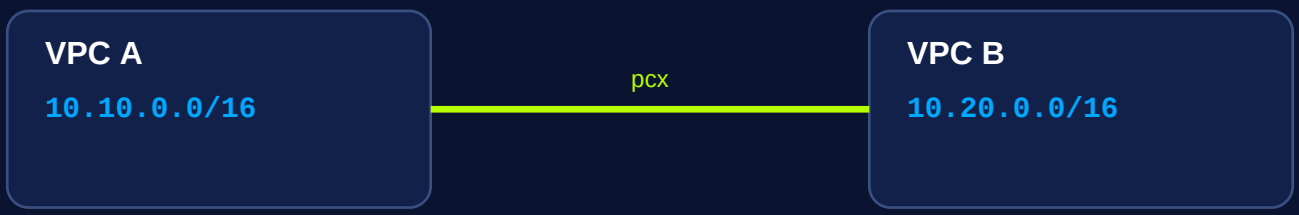
BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs. Prove private connectivity.

EC2, custom route tables, security groups and a bastion host (ProxyJump)



AWS ARCHITECT ASSOCIATE

Name:	Kakkireni Abhinav
Batch:	AWS Internship Batch 1
Region:	ap-south-1 (Mumbai)
Completion date:	29-10-2026
Project:	Project 01 / VPC Peering

1. Project overview

This project connects two AWS VPCs with a VPC peering connection so that two private EC2 instances can exchange traffic over their private IPv4 addresses. A single public bastion host in VPC A is used for administration through SSH ProxyJump. No NAT gateway is used and the private instances have no public IP addresses.

What was demonstrated

- Non-overlapping VPC and subnet CIDR planning (10.10.0.0/16 and 10.20.0.0/16).
- Public and private subnets with explicit route table associations.
- An accepted (Active) peering connection with routes on both sides and scoped security group rules.
- A failed baseline test before peering, successful two-way tests after peering, and a controlled failure with recovery.
- Full cleanup of every resource created for the project.

Architecture

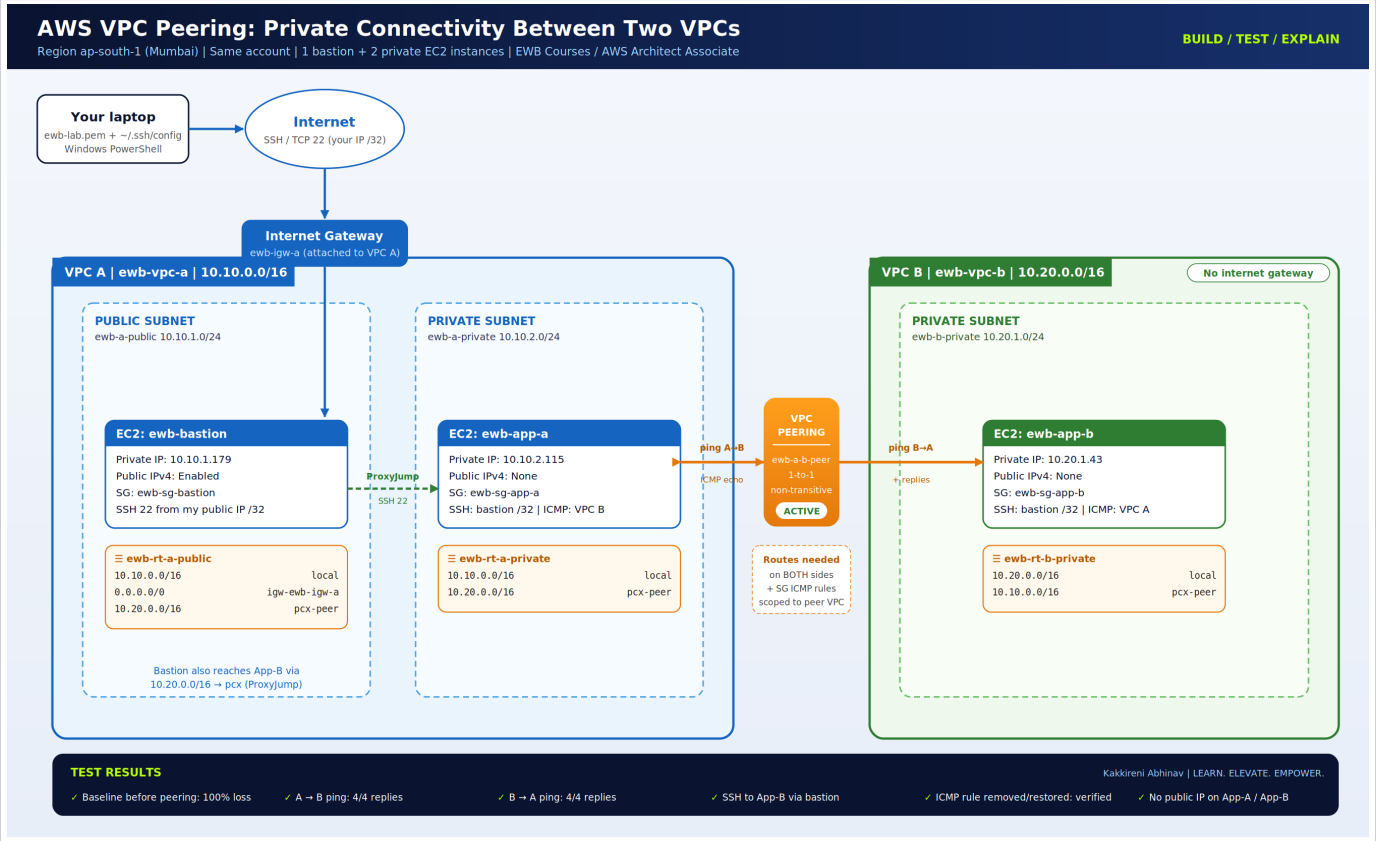


Figure 1. Architecture of the project as built (actual private IPs and resource names).

Administration path: laptop to bastion to App-A or App-B using SSH ProxyJump. Project traffic path: App-A and App-B over private IPs through the peering connection. Peering does not give VPC B internet access through VPC A.

2. Resource inventory

Resource	Name tag	CIDR / private IP	AWS ID
VPC A	ewb-vpc-a	10.10.0.0/16	`vpc-01c24171c6c04cc82`
Public subnet A	ewb-a-public	10.10.1.0/24 (ap-south-1a)	`subnet-0fbadaedf92fefe30`
Private subnet A	ewb-a-private	10.10.2.0/24 (ap-south-1a)	`subnet-0a9d23bff3356f78d`
VPC B	ewb-vpc-b	10.20.0.0/16	`vpc-0bc004e8379748903`
Private subnet B	ewb-b-private	10.20.1.0/24 (ap-south-1a)	`subnet-077a33a39cbd8e284`
Internet gateway	ewb-igw-a	Attached to VPC A	`igw-0d84c5a0598c751f8`
Route table (public A)	ewb-rt-a-public	Associated: ewb-a-public	`rtb-02946d1c390072289`
Route table (private A)	ewb-rt-a-private	Associated: ewb-a-private	`rtb-06d32d84998dbb227`
Route table (private B)	ewb-rt-b-private	Associated: ewb-b-private	`rtb-07a3c4c190ce9bef9`
Peering connection	ewb-a-b-peer	VPC A (requester) to VPC B (accepter)	`pcx-0cedff64a2a507446`
Bastion host	ewb-bastion	10.10.1.179 + public IPv4	`i-00ad0715e8f7c97d0`
App server A	ewb-app-a	10.10.2.115 / no public IP	`i-0abcd0b167da0734b`
App server B	ewb-app-b	10.20.1.43 / no public IP	`i-05281172204aa8eb7`
Security group	ewb-sg-bastion	VPC A	`sg-0970e0cb5393d4abe`
Security group	ewb-sg-app-a	VPC A	`sg-068d2b99dad3e5448`
Security group	ewb-sg-app-b	VPC B	`sg-0f97fd2c9ae951c2e`
Key pair	ewb-lab2 (RSA, .pem kept on laptop only)	-	-

All instances are Amazon Linux 2023 on t2.micro. All resources are in one account and one Region (ap-south-1).

3. Routing and security design

Route tables

Route table	Destination	Target	Purpose
ewb-rt-a-public	`10.10.0.0/16`	`local`	Bastion to App-A
ewb-rt-a-public	`0.0.0.0/0`	`igw-0d84c5a0598c751f8`	Bastion internet access
ewb-rt-a-public	`10.20.0.0/16`	`pcx-0cedff64a2a507446`	Bastion SSH to App-B
ewb-rt-a-private	`10.10.0.0/16`	`local`	VPC A local
ewb-rt-a-private	`10.20.0.0/16`	`pcx-0cedff64a2a507446`	App-A to VPC B (forward path)
ewb-rt-b-private	`10.20.0.0/16`	`local`	VPC B local
ewb-rt-b-private	`10.10.0.0/16`	`pcx-0cedff64a2a507446`	Return path to VPC A

Security group inbound rules

Security group	Type / port	Source
ewb-sg-bastion	SSH / TCP 22	My public IPv4 /32
ewb-sg-app-a	SSH / TCP 22	10.10.1.179/32 (bastion private IP)
ewb-sg-app-a	Custom ICMP IPv4 - Echo Request	10.20.0.0/16 (VPC B)
ewb-sg-app-b	SSH / TCP 22	10.10.1.179/32 (bastion private IP)
ewb-sg-app-b	Custom ICMP IPv4 - Echo Request	10.10.0.0/16 (VPC A)

Outbound rules and network ACLs were left at the default allow-all. Security groups are stateful, so replies to permitted traffic are allowed automatically.

4. Build evidence

Network: VPCs, subnets, internet gateway

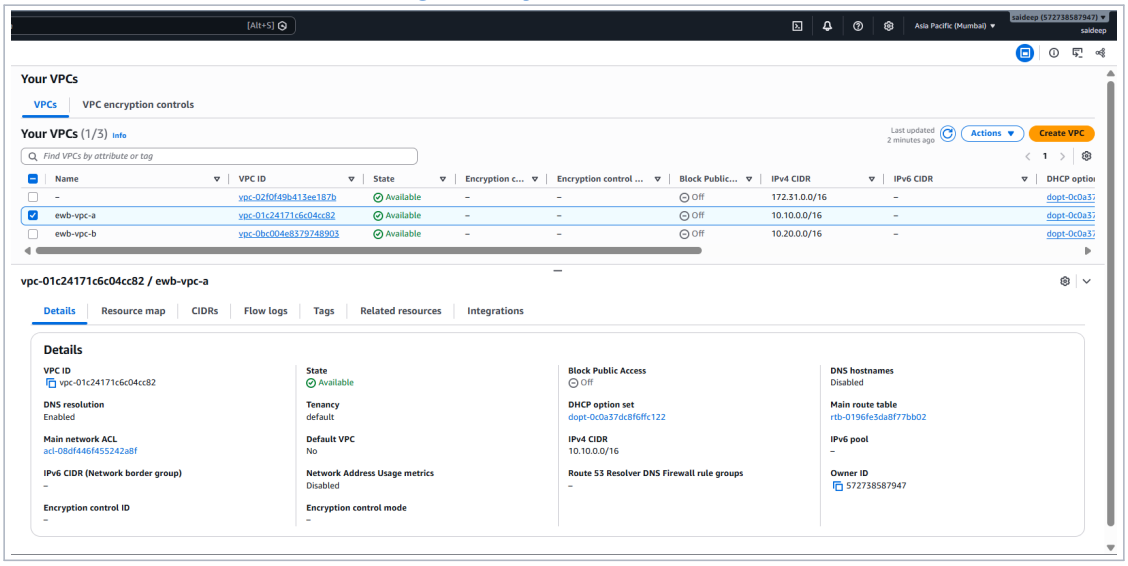


Figure 2. Both VPCs with non-overlapping CIDRs: ewb-vpc-a 10.10.0.0/16 and ewb-vpc-b 10.20.0.0/16.

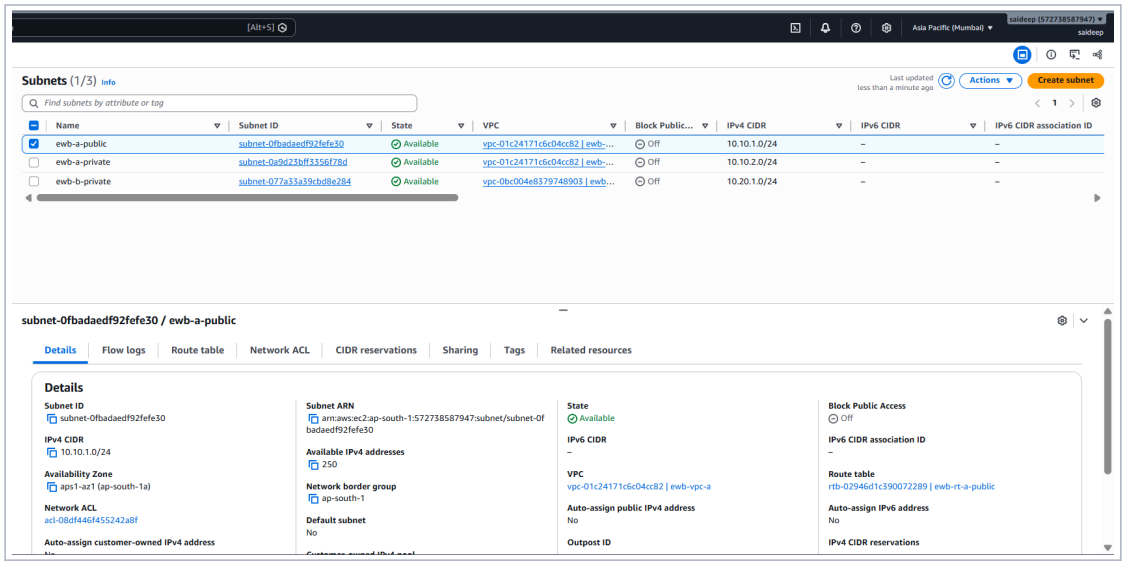


Figure 3. Subnets ewb-a-public 10.10.1.0/24, ewb-a-private 10.10.2.0/24 and ewb-b-private 10.20.1.0/24. ewb-a-public is explicitly associated with ewb-rt-a-public and auto-assign public IPv4 is No.

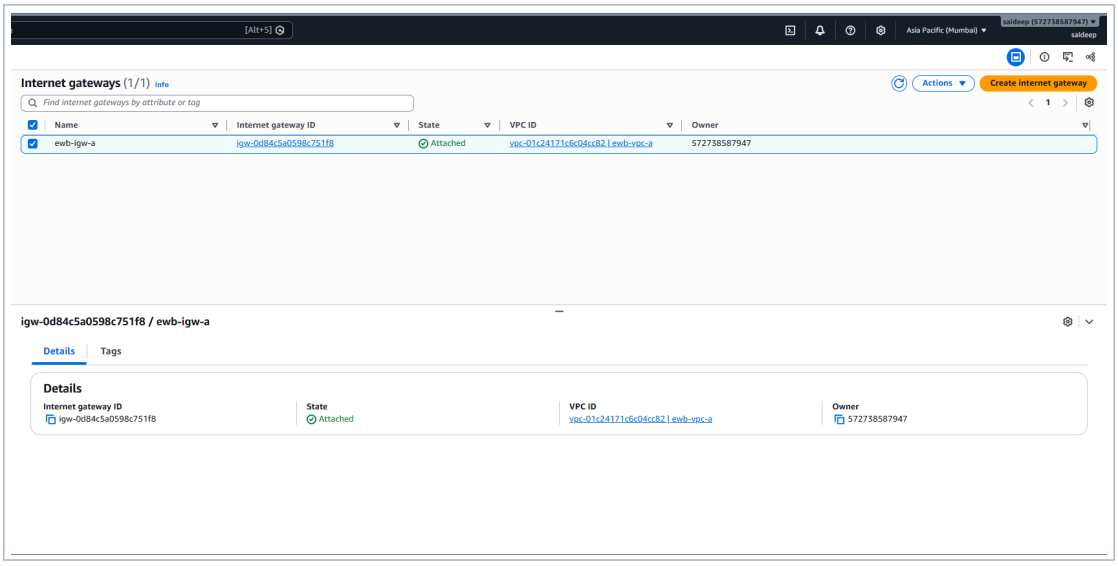


Figure 4. Internet gateway ewb-igw-a is Attached to ewb-vpc-a.

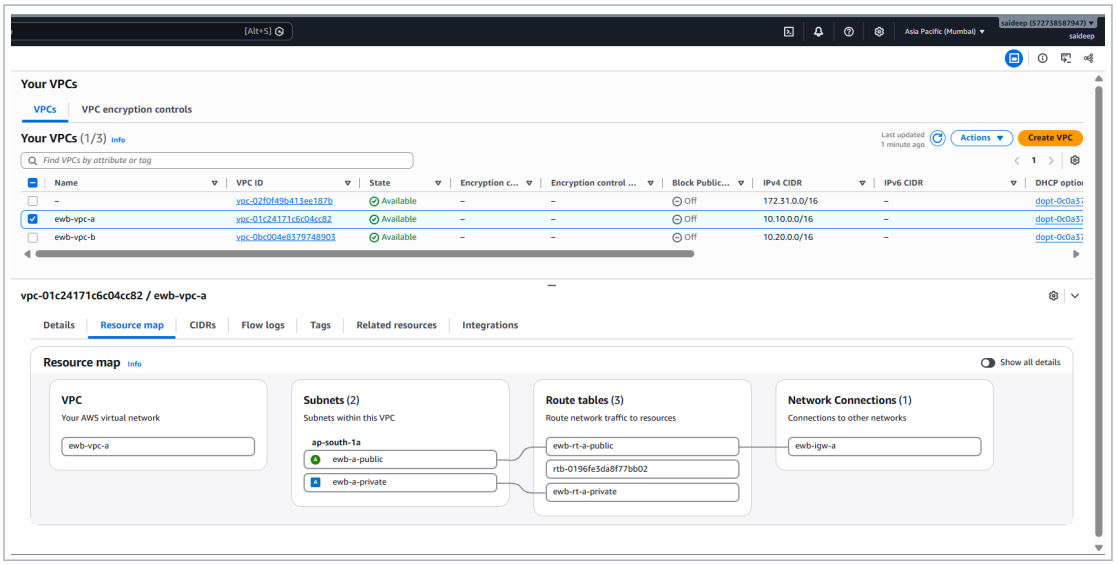


Figure 5. Resource map of ewb-vpc-a: public and private subnets, route tables, and the internet gateway.

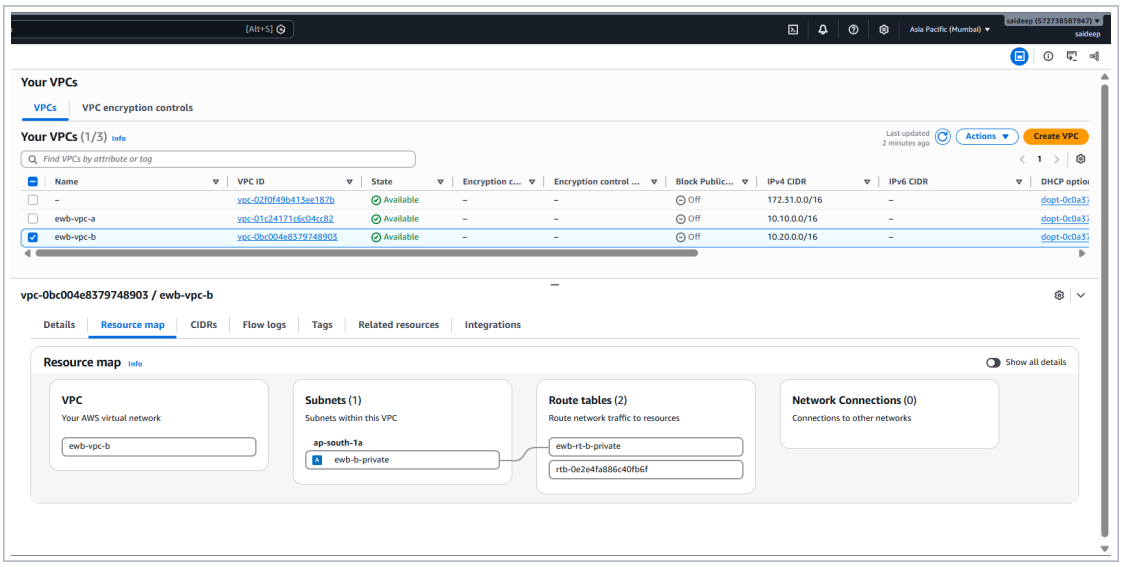


Figure 6. Resource map of ewb-vpc-b: one private subnet, its own route table and no network connections (no internet gateway).

Instances

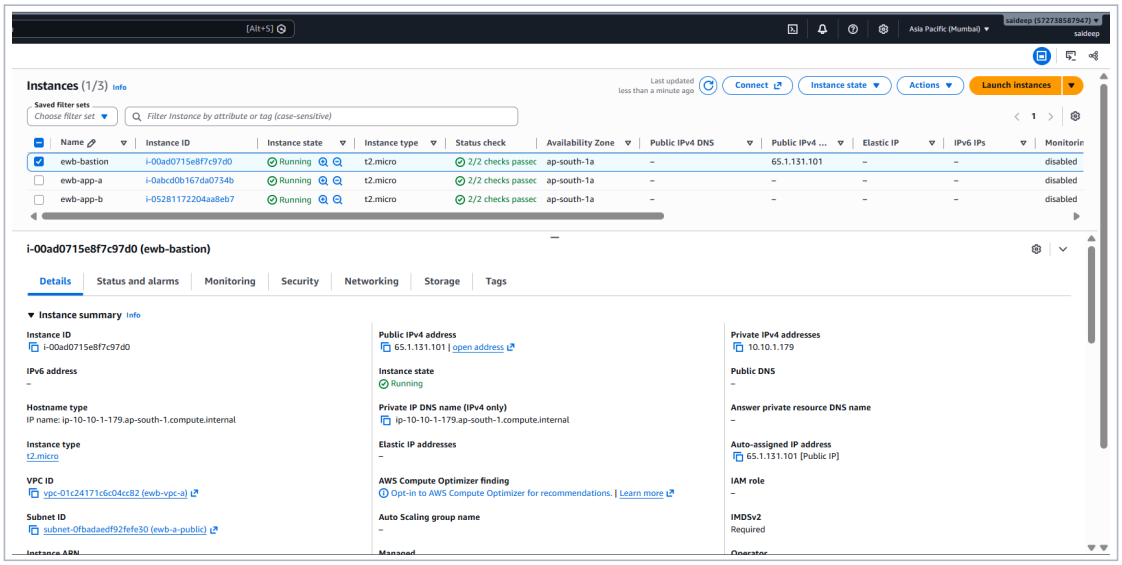


Figure 7. ewb-bastion (i-00ad0715e8f7c97d0): private IP 10.10.1.179, public IPv4 assigned, in ewb-a-public, 2/2 status checks passed.

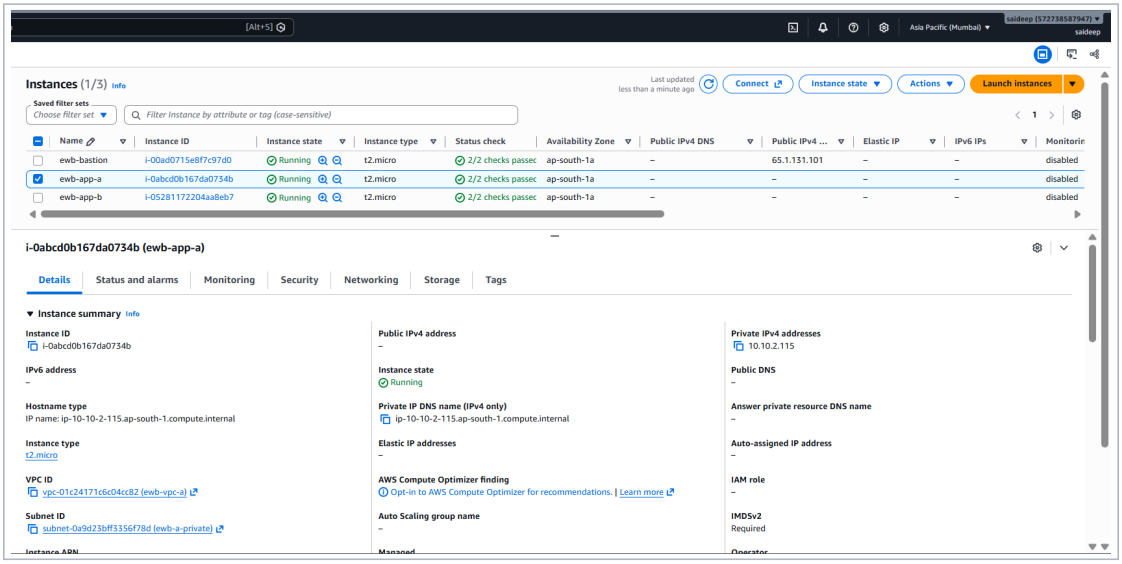


Figure 8. ewb-app-a (i-0abcd0b167da0734b): private IP 10.10.2.115, Public IPv4 address empty, in ewb-a-private.

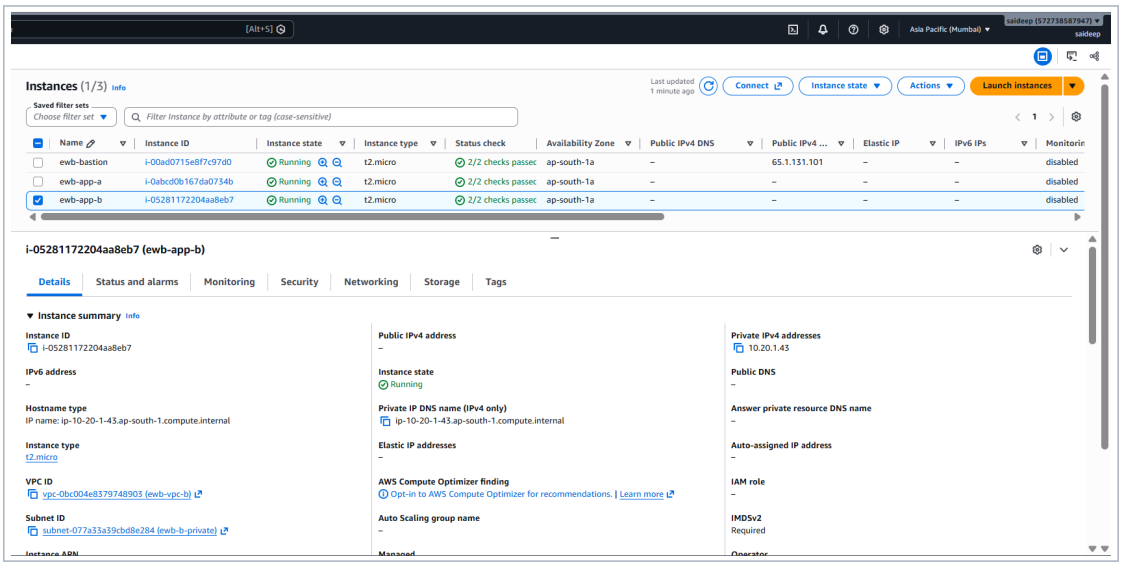


Figure 9. ewb-app-b (i-05281172204aa8eb7): private IP 10.20.1.43, Public IPv4 address empty, in ewb-b-private (VPC B).

Peering and routes

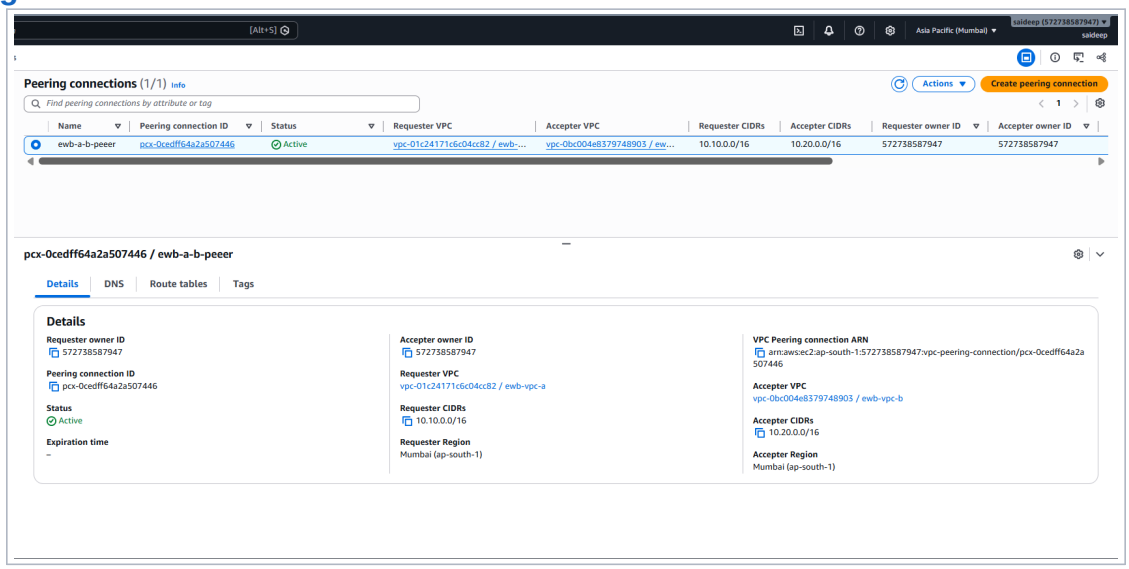


Figure 10. Peering connection pcx-0cedff64a2a507446 (ewb-a-b-peer) is Active. Requester VPC A 10.10.0.0/16, accepter VPC B 10.20.0.0/16, same account and Region.

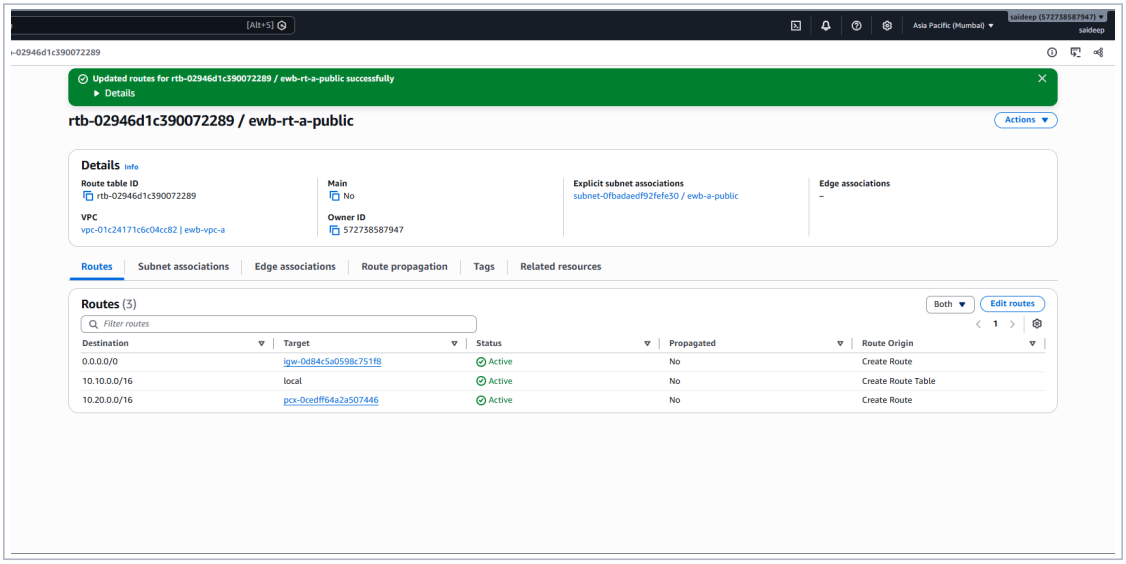


Figure 11. ewb-rt-a-public: 0.0.0.0/0 to the internet gateway, 10.10.0.0/16 local, 10.20.0.0/16 to the peering connection. Explicit association: ewb-a-public.

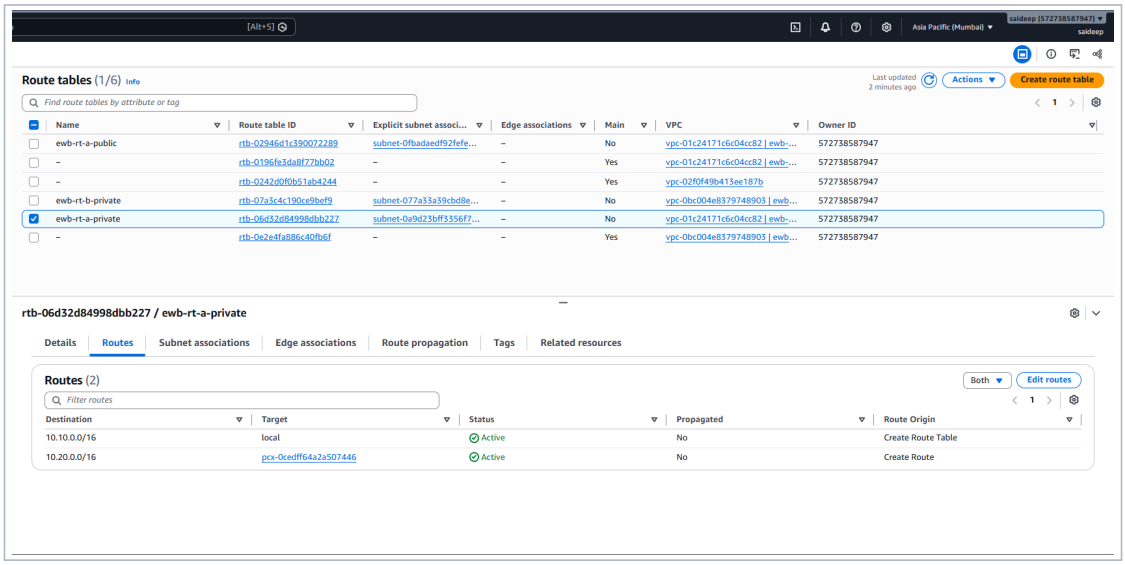


Figure 12. ewb-rt-a-private: local and 10.20.0.0/16 to the peering connection (forward path). No internet route.

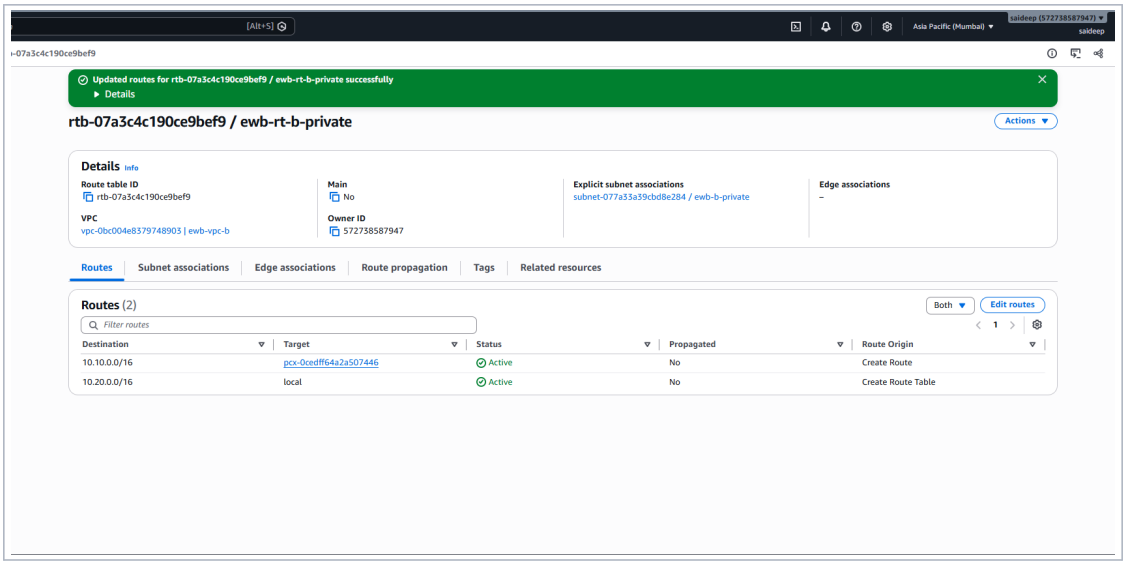


Figure 13. ewb-rt-b-private: 10.10.0.0/16 to the peering connection (return path) and 10.20.0.0/16 local. Explicit association: ewb-b-private.

Security groups

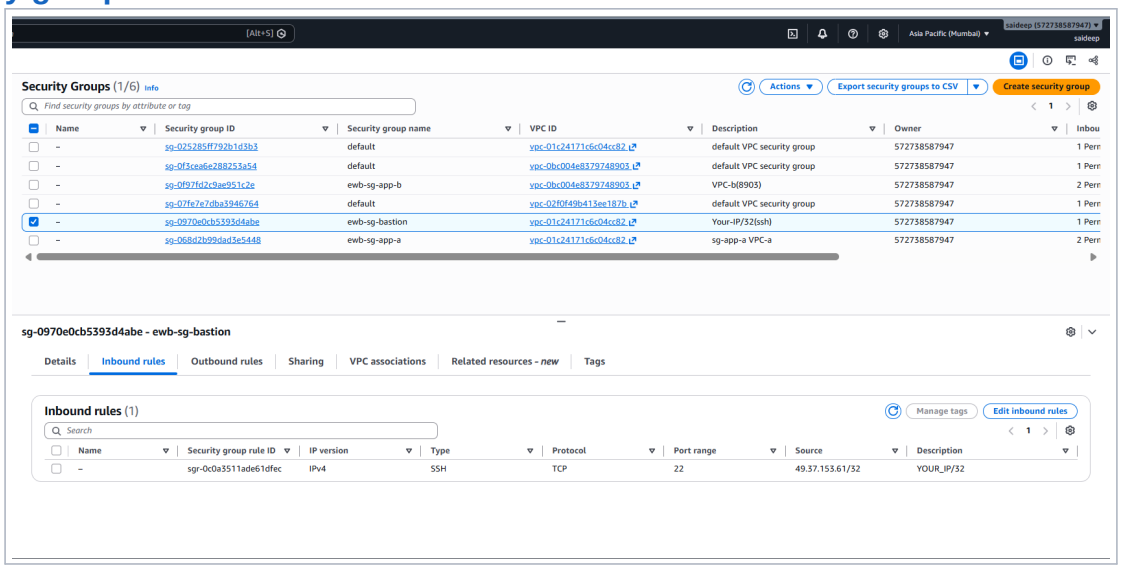


Figure 14. ewb-sg-bastion: SSH TCP 22 allowed only from my public IPv4 /32.

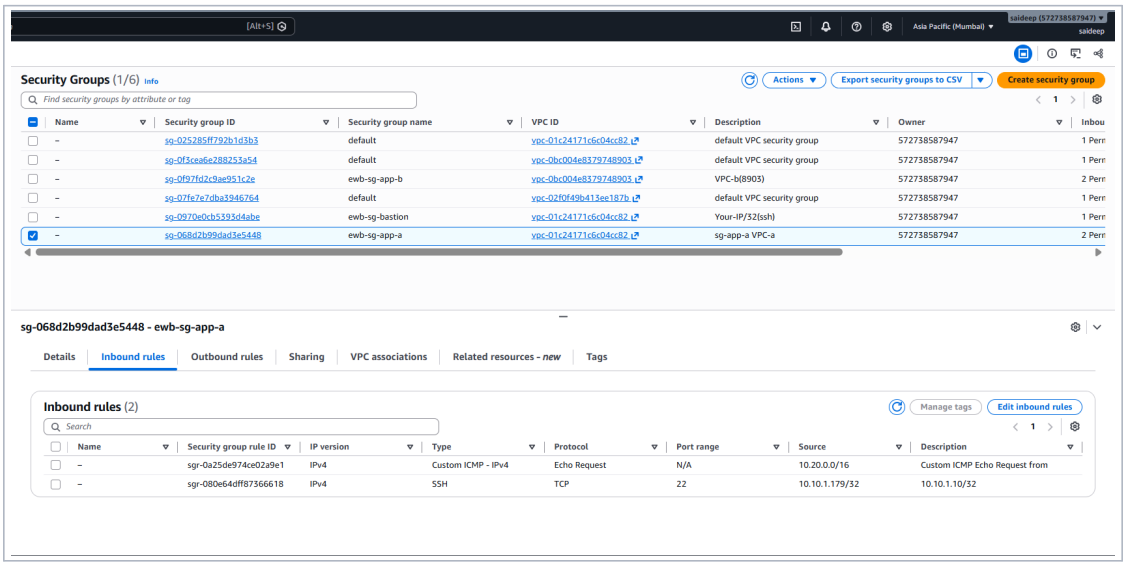


Figure 15. ewb-sg-app-a: ICMP Echo Request from VPC B (10.20.0.0/16) and SSH from the bastion private IP 10.10.1.179/32.

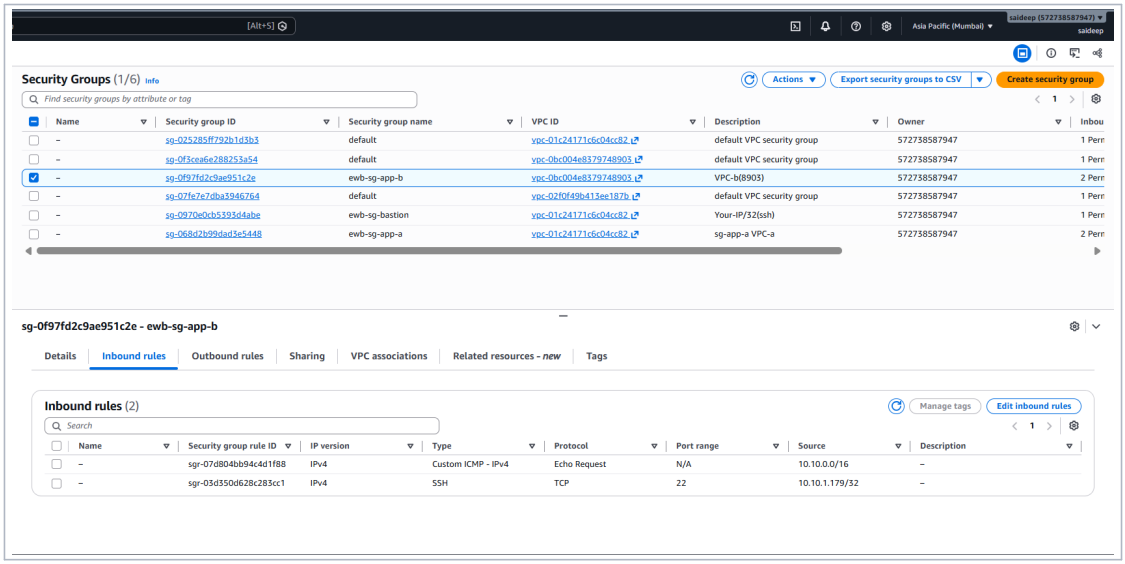


Figure 16. ewb-sg-app-b: ICMP Echo Request from VPC A (10.10.0.0/16) and SSH from the bastion private IP 10.10.1.179/32.

5. Test evidence

Test	Where / command	Result	Evidence
01 Baseline	App-A, before peering: ping -c 4 10.20.1.10 and 10.20.1.43	No replies, 100% loss	Figure 17
02 Forward path	App-A, after routes: ping -c 4 10.20.1.43	4/4 replies, 0% loss	Figure 18
03 Reverse path	App-B: ping -c 4 10.10.2.115	4/4 replies, 0% loss	Figure 19
04 Private SSH	Laptop: ssh ewb-app-b (via bastion ProxyJump)	Shell on App-B	Figure 19
05 Isolation	EC2 instance details of App-A and App-B	No public IPv4	Figure 8, Figure 9
06 Controlled failure	App-A: ping App-B with its ICMP Echo rule removed	100% loss	Figure 18
07 Recovery	ICMP rule restored, ping again	4/4 replies, 0% loss	Figure 18

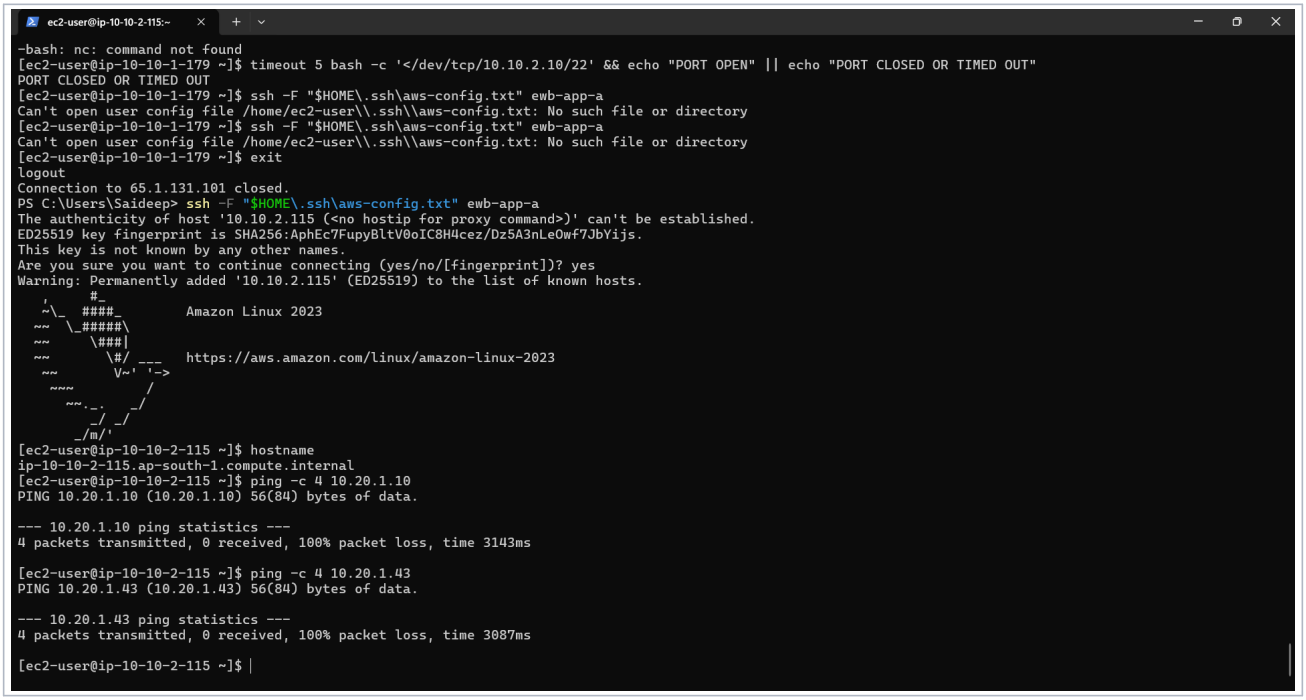


Figure 17. BASELINE (before peering, 01:43): from App-A (10.10.2.115) pings to 10.20.1.10 and to App-B's real address 10.20.1.43 both show 100% packet loss.

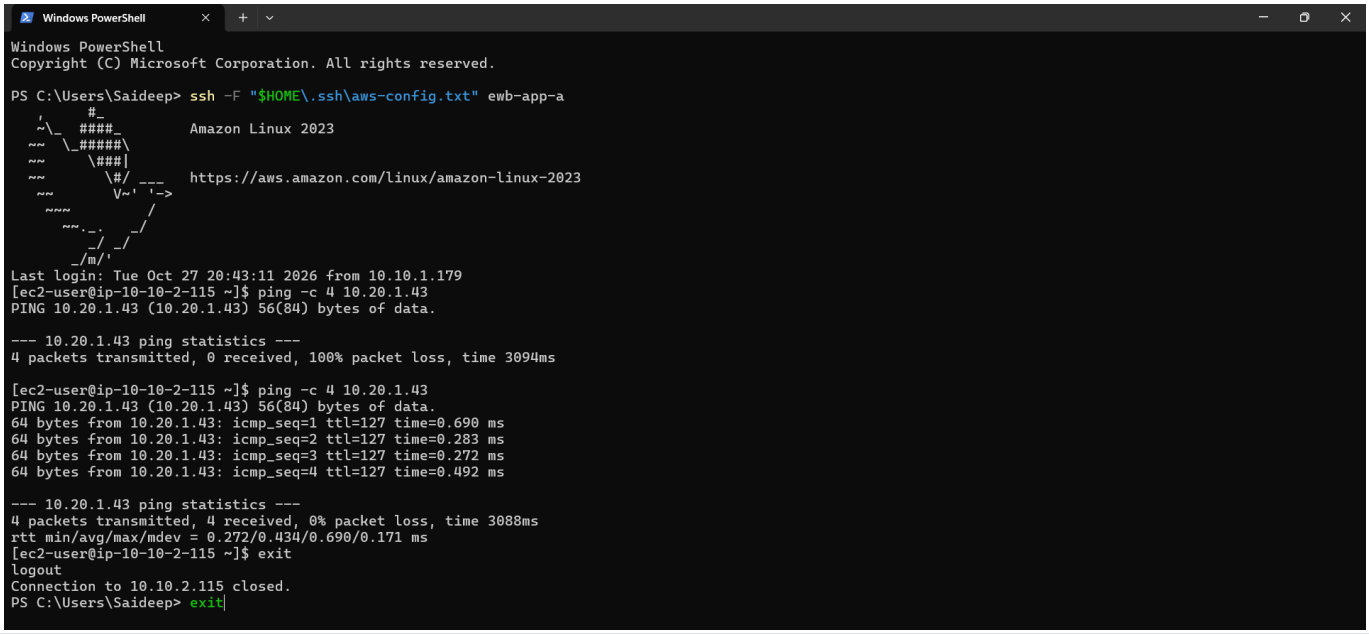


Figure 18. CONTROLLED FAILURE AND RECOVERY: with the App-B inbound ICMP Echo rule removed, the first ping from App-A to 10.20.1.43 shows 100% packet loss. After the rule was restored, the second ping returns 4/4 replies, 0% loss (forward path).

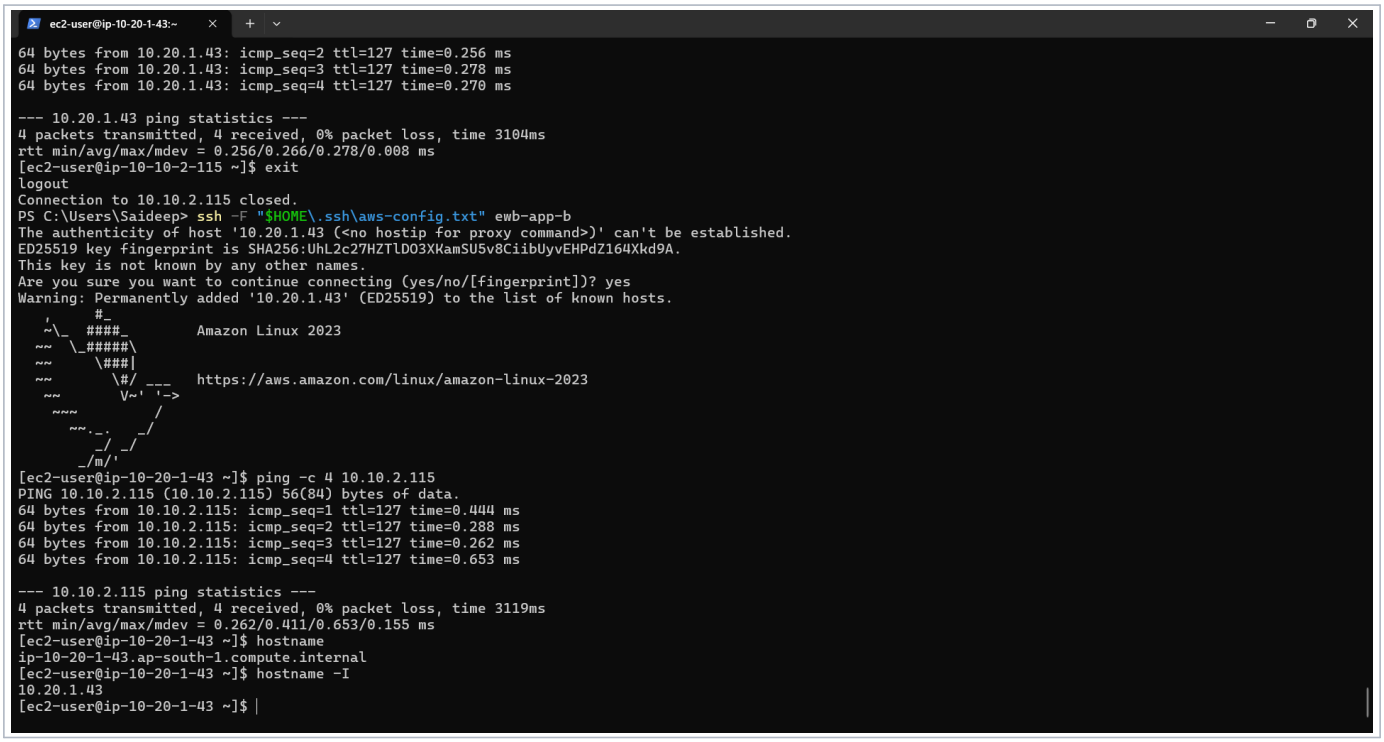


Figure 19. REVERSE PATH AND PRIVATE SSH: connected to App-B through the bastion (ssh ewb-app-b), hostname ip-10-20-1-43, hostname -I 10.20.1.43, and ping -c 4 10.10.2.115 returns 4/4 replies, 0% loss.

6. How it works: forward and return path

Forward path. App-A (10.10.2.115) sends an ICMP echo request to App-B (10.20.1.43). The subnet ewb-a-private uses ewb-rt-a-private. The destination does not match the local 10.10.0.0/16 route, but it matches 10.20.0.0/16, whose target is the peering connection pcx-0cedff64a2a507446. The packet crosses the peering connection into VPC B, where ewb-sg-app-b allows ICMP Echo Request from VPC A (10.10.0.0/16), so App-B receives it.

Return path. App-B replies to 10.10.2.115. The subnet ewb-b-private uses ewb-rt-b-private, where 10.10.0.0/16 points to the same peering connection, so the reply travels back to VPC A. Security groups are stateful, so the reply needs no inbound rule on App-A. Without the return route the replies have no path and the ping shows 100% loss.

Administration path. My laptop connects to the bastion public IP on TCP 22 (allowed only from my IP /32). With ProxyJump, the bastion opens a new SSH connection to App-A or App-B, so those hosts see the bastion's private IP

10.10.1.179 as the source. That is why their SSH rules use 10.10.1.179/32 and why ewb-rt-a-public needs a 10.20.0.0/16 route to the peering connection.

7. Deviations from the workbook (and why)

Item	Workbook	What I built	Impact
Private IP addresses	10.10.1.10, 10.10.2.10, 10.20.1.10	AWS-assigned: 10.10.1.179, 10.10.2.115, 10.20.1.43	The workbook allows this if all IPs and rules are updated. Security group rules use the real addresses. My first baseline ping targeted 10.20.1.10, so I repeated it against 10.20.1.43.
ICMP security group source	App-A /32 and App-B /32	Peer VPC CIDRs (10.10.0.0/16 and 10.20.0.0/16)	Still restricted to the peer VPC only, never 0.0.0.0/0, but wider than a single host. A /32 per host would be stricter and is the better practice.
Instance type	t3.micro (example)	t2.micro	Same small x86 class, lab use only.
Key pair	ewb-lab	ewb-lab2	The first key did not match the instances (see troubleshooting).
SSH config	~/.ssh/config	aws-config.txt used with ssh -F	Same entries, ProxyJump through the bastion.

8. Troubleshooting log

The first bastion attempt failed with Permission denied (publickey). Routing was working because the connection reached SSH authentication; the problem was the key (ewb-lab.pem did not match the instance key pair). The fix was to use the correct key (ewb-lab2.pem). Windows then warned that the key file permissions were too open (UNPROTECTED PRIVATE KEY FILE); this was fixed with icacls (remove inheritance, grant read to my user only), after which the bastion login worked.

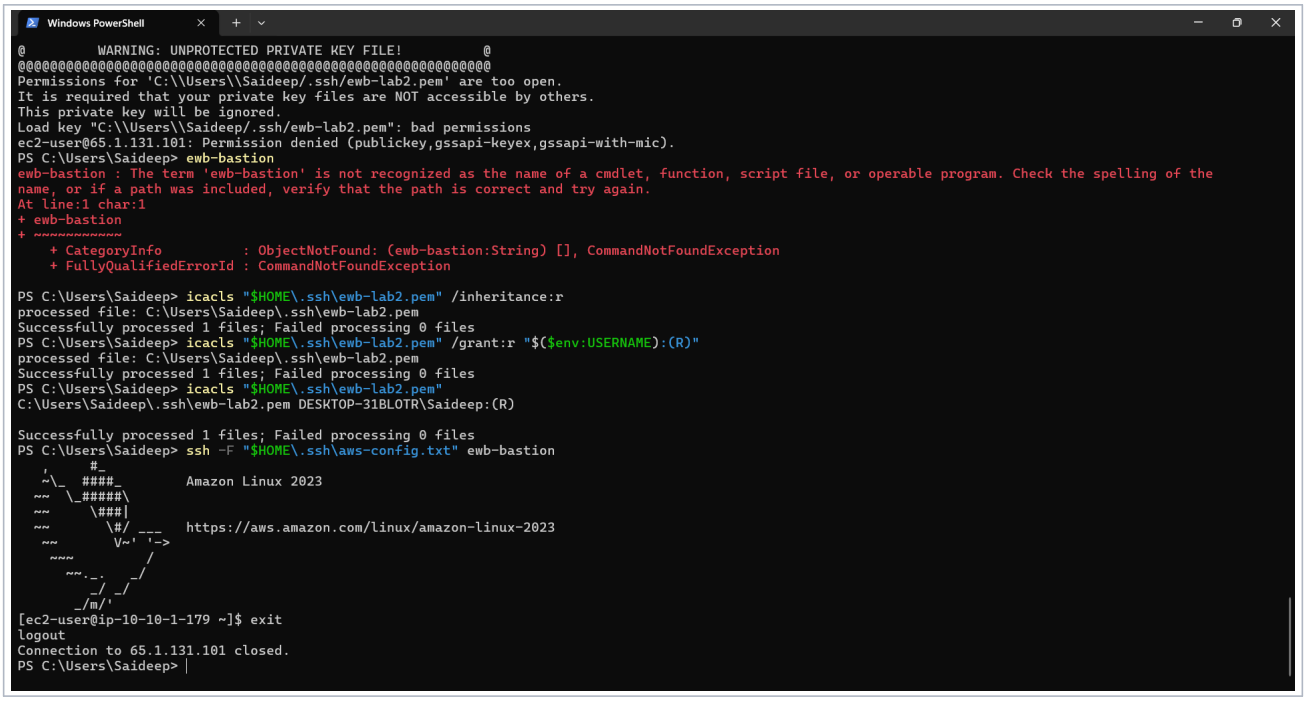


Figure 20. Bastion login after the key permissions were fixed with icacls. Prompt shows ip-10-10-1-179.

9. Cleanup

All resources created for this project were removed on 29-10-2026 in dependency order. The default VPC (172.31.0.0/16) was left untouched.

Step	Resource	Status	Evidence
1	Instances ewb-bastion, ewb-app-a, ewb-app-b	Terminated	Figure 21
2	Peer routes and peering connection ewb-a-b-peer	Deleted (3 of 3 related routes removed)	Figure 22
3	Security groups ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b	Removed before the VPCs were deleted (no separate screenshot)	Figure 26
4	Subnets and custom route tables	Deleted	Figure 23, Figure 24
5	Internet gateway ewb-igw-a	Detached, then deleted	Figure 25
6	VPCs ewb-vpc-a and ewb-vpc-b	Deleted; only the default VPC remains	Figure 26

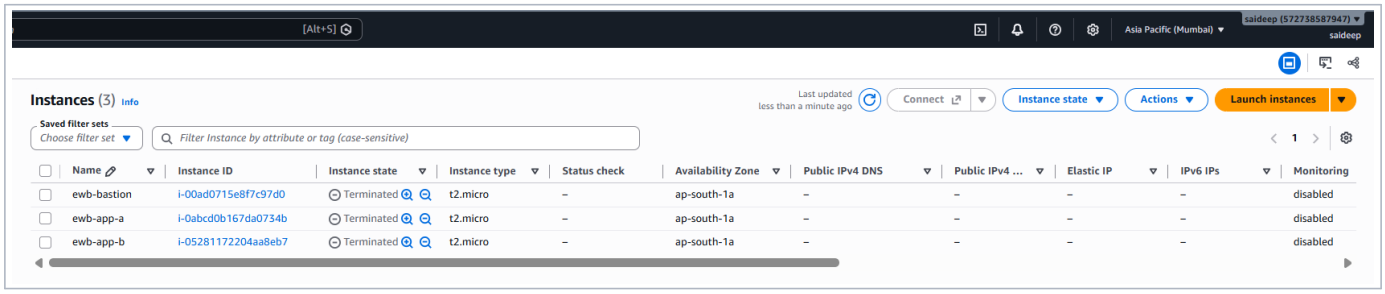


Figure 21. All three instances show Terminated.

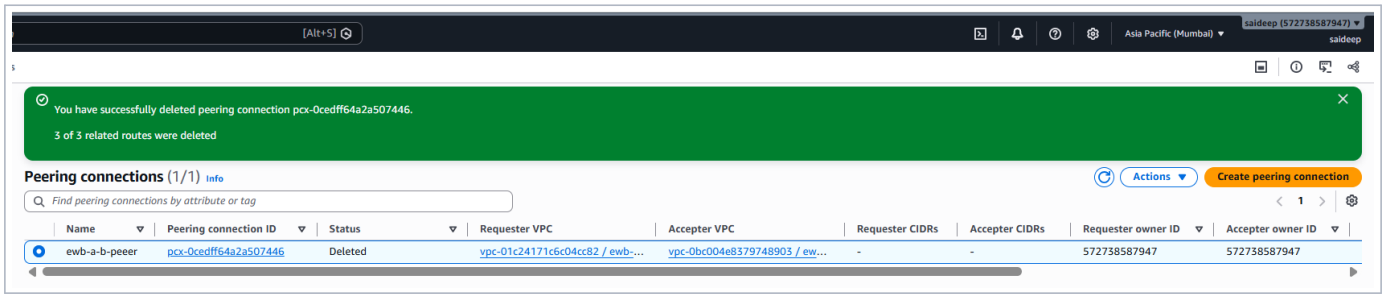


Figure 22. Peering connection pcx-0cedff64a2a507446 deleted; 3 of 3 related routes deleted (09:32).

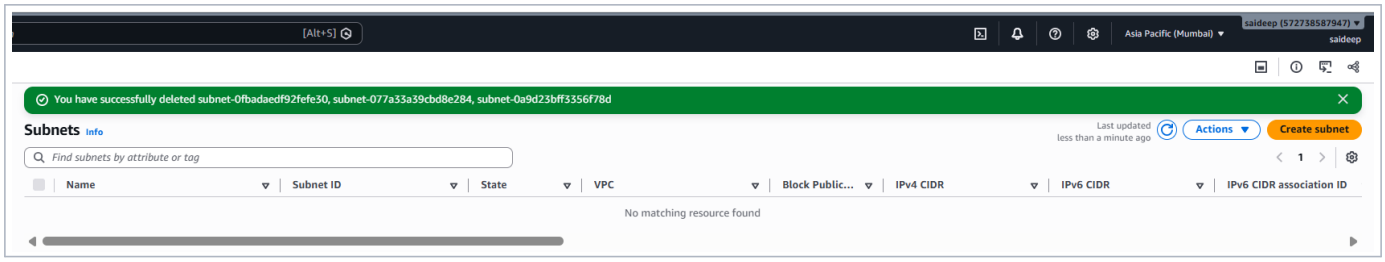


Figure 23. Subnets deleted: no matching resource found (09:33).

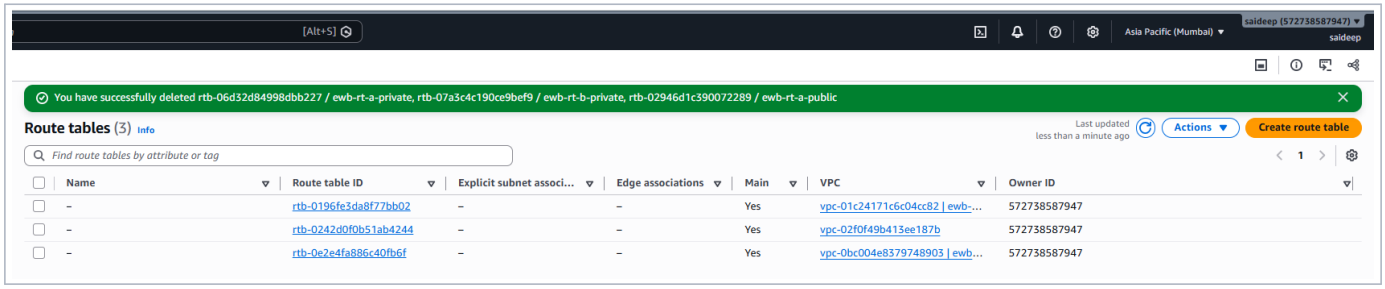


Figure 24. Custom route tables ewb-rt-a-private, ewb-rt-b-private and ewb-rt-a-public deleted; only the three main (default) route tables remain (09:34).

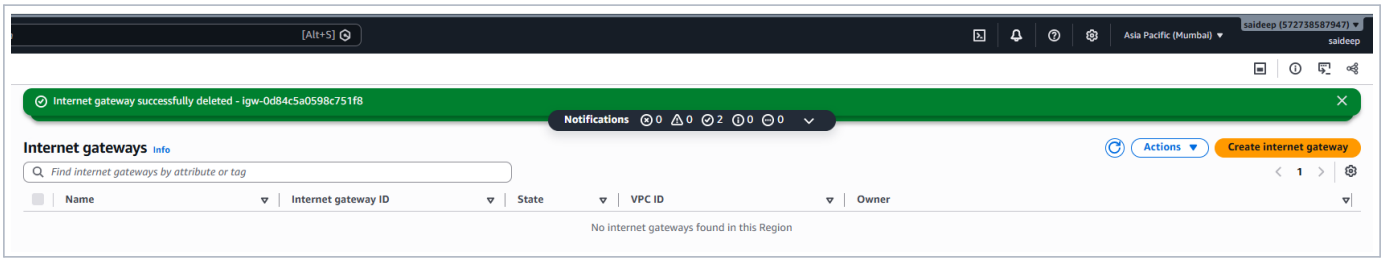


Figure 25. Internet gateway igw-0d84c5a0598c751f8 deleted after being detached from the VPC: no internet gateways found in this Region (09:35).

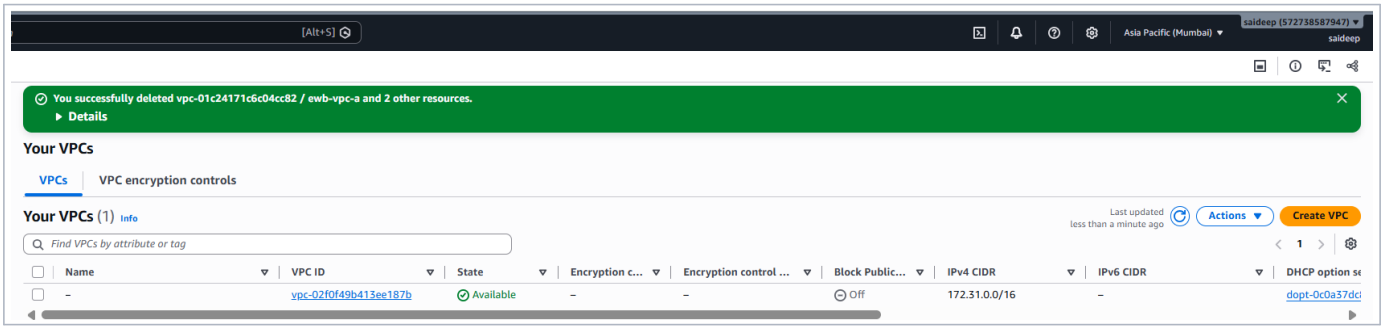


Figure 26. Both lab VPCs deleted. Only the default VPC 172.31.0.0/16 remains (09:37).

10. What I learned

- VPC CIDR ranges must not overlap for peering; different subnet ranges alone do not fix overlapping VPC ranges.
- An Active peering connection does nothing by itself. Routes are needed on both sides, and security groups must allow the traffic.
- Security groups are stateful, but route tables are not: the return route has to exist.
- A bastion host with ProxyJump gives access to private instances without copying private keys onto servers, and the private hosts see the bastion's private IP as the source.
- Peering is one-to-one and non-transitive, and does not provide internet access to the peer VPC.
- Reading the error (timeout vs Permission denied (publickey) vs key permission warning) quickly shows whether the problem is networking or authentication.
- Use the real assigned IP addresses in tests and security group rules, and clean up all resources after the lab to avoid charges.